A One-Dimensional Neural Cellular Automaton for Next-Tool Prediction in Agent Traces

Julian M. Kleber
Independent Researcher
ORCID: 0000-0001-5518-0932
julian.m.kleber@gmail.com

October 2026

Abstract

Autonomous agent execution loops leave behind structured traces of tool calls, arguments, and environment responses. Routing the next action in these loops typically relies on autoregressive Large Language Models (LLMs), which impose significant inference latency, token expenditure, and memory overheads. In this work, we investigate whether local, iterative update rules via a One-Dimensional Neural Cellular Automaton (**TextNCA**) can effectively perform next-tool prediction within agent traces under bounded compute. TextNCA operates over a compact 1D cell grid representing the user goal and the most recent execution steps ($L = 5$), updating a 16-channel hidden state through depthwise 1D spatial perception and gated recurrent residual updates. To avoid under-trained comparison traps common in iterative dynamics, all architectural claims adhere to a 150,000-step training floor. On curated agent execution traces under a 13-label vocabulary, TextNCA achieves 65.18% top-1 accuracy, outperforming equal-budget flat linear readouts (60.71%), constant always-terminal policies (54.30%), and small local LLMs (Qwen2.5-1.5B-Instruct at 6.29%). Systematic 150k-step ablations demonstrate that long-range goal broadcast channels achieve 63.55% top-1 accuracy on complex sub-tasks, establishing cellular automata as an ultra-compact, high-throughput routing layer for multi-agent systems.

1 Introduction

Modern autonomous agent architectures iteratively cycle through perceiving environment observations, reasoning over goals, and dispatching tool invocations. While massive frontier models dictate high-level planning, executing intermediate tool routing via multi-billion parameter autoregressive decoders introduces severe latency bottlenecks, context-window bloat, and operational cost.

Neural Cellular Automata (NCAs) [**?**] have shown remarkable capability in learning local, self-organizing update rules for image morphogenesis, pattern regeneration, and spatial computing. However, NCAs have primarily remained restricted to 2D continuous lattices. While recent explorations have probed text-level NCAs, their applicability to discrete tool-routing graphs in agent traces has not been systematically established.

In this paper, we propose a discrete one-dimensional Neural Cellular Automaton (**TextNCA**) designed specifically for next-tool prediction in agent sessions. We demonstrate that:

1. A 1D NCA with $H = 16$ hidden channels and depthwise spatial message passing effectively routes agent actions across a 13-tool vocabulary.
2. Under a rigorous $\geq$ 150,000 AdamW training floor, TextNCA outperforms flat linear baselines (65.18% vs. 60.71%) and compact local language models (6.29%).
3. We isolate the contribution of state carry, soft halting, and long-range goal broadcast channels through equal-budget 150k ablation studies.

2 Methodology

2.1 Task Definition and Trace Representation

At each decision turn in an agent session, the model observes the current state and predicts the next tool identifier $y \in \mathcal{V}$, where $|\mathcal{V}| = 13$ covers standard agent capabilities (file reads, edits, command execution, and terminal session completion).

The observation is mapped to a 1D sequence of length $L = 5$:

- **Cell 0:** The root user objective/goal.
- **Cells 1–4:** The four most recent interaction steps, right-aligned, containing the previous tool ID and truncated output hash.

Each cell comprises a 48-dimensional hashed bag-of-tokens representation, a one-hot encoding of the prior tool, and an active occupancy bit. Observation channels remain immutable during the cellular update.

2.2 TextNCA Architecture

Let $h_i^{(t)} \in \mathbb{R}^H$ denote the hidden state of cell $i \in \{0, \ldots, L-1\}$ at update step $t \in \{0, \ldots, K-1\}$, initialized to zero. Perception is computed via a 1D convolution bank combining identity, spatial gradient, and 1D discrete Laplacian filters:

$$p_i^{(t)} = \left[h_i^{(t)},\ h_{i+1}^{(t)} - h_{i-1}^{(t)},\ h_{i+1}^{(t)} - 2h_i^{(t)} + h_{i-1}^{(t)} \right] \quad (1)$$

The perceived state is concatenated with the cell's fixed observation features x_i and passed through a shared 1×1 pointwise update network:

$$\Delta h_i^{(t)} = W_2 \, \mathrm{GELU}(W_1 [p_i^{(t)}; x_i] + b_1) + b_2 \quad (2)$$

The update is gated via a stochastic or learned update mask $m_i^{(t)} \in [0, 1]$:

$$h_i^{(t+1)} = h_i^{(t)} + m_i^{(t)} \odot \Delta h_i^{(t)} \quad (3)$$

After $K = 4$ internal propagation steps, a linear readout layer maps the flattened final hidden state $\mathrm{vec}(H^{(K)})$ to tool class logits.

2.3 Architectural Variants and Ablations

To dissect the mechanics of 1D agent message passing, we formulate five distinct variants:

- **Baseline:** Resets hidden channels $h_i^{(0)} = 0$ on every turn.
- **Long Range:** Broadcasts the goal embedding x_0 into the perception of cells $1 \ldots 4$ via a zero-initialized learned projection.
- **Carry:** Carries the hidden state $h^{(K)}$ forward into the initial state $h^{(0)}$ of the next decision step within the same session (truncated BPTT).
- **Halt:** A learned soft halting module that dynamically terminates updates when state delta magnitudes drop below threshold ϵ.
- **Flat:** An equal-parameter feed-forward linear baseline operating directly on the flattened observation vector.

3 Experimental Setup

Training Protocol. Models are optimized using AdamW with an initial learning rate of 10^{-3}, cosine decay to 1%, weight decay of 10^{-2}, and gradient norm clipping at 1.0. The loss function is class-weighted cross-entropy with inverse-frequency weighting. Crucially, all comparative runs enforce a minimum of 150,000 steps to avoid premature conclusions drawn from early learning dynamics.

Baselines. We compare against:

1. **Always-Terminal:** A majority-class heuristic that unconditionally predicts the session-ending action.
2. **Flat Twin:** An equal-capacity feedforward linear network.
3. **Autoregressive LLM:** Qwen2.5-1.5B-Instruct evaluated in zero-shot/few-shot greedy decoding on identical tokenized inputs.

4 Results

4.1 Primary Next-Tool Classification

Table **??** presents the primary benchmark results on curated agent traces.

Table 1: Primary evaluation on curated agent traces (13-tool vocabulary). Evaluated under 150k AdamW step budget.

Model / Setting	Experiment ID	Val Top-1	Local Top-1
TextNCA (Mix13)	`EXP-20261003-001`	**65.18%**	40.09%
Flat Linear Twin	`EXP-20261003-001`	60.71%	37.37%
FineEnvs Continuation	`EXP-20261005-001`	58.41%	40.33%
Always-Terminal	Baseline	54.30%	**49.47%**
Qwen2.5-1.5B-Instruct	`EXP-20261004-001`	—	6.29%

TextNCA achieves 65.18% validation accuracy on the curated benchmark, establishing a +4.47% margin over the flat linear baseline (60.71%) and +10.88% over the always-terminal heuristic. In zero-shot evaluation on local unseen developer traces, Qwen2.5-1.5B collapses to 6.29% due to formatting sensitivity on closed vocabulary extraction, whereas TextNCA maintains robust routing behavior.

4.2 Ablation Analysis at 150k Steps

Table **??** details the 150k-step ablations isolating the architectural components of the 1D update rule.

Table 2: Systematic 150k-step ablation study on message passing and memory mechanisms.

Variant	Primary EXP	Val Top-1	Best Step
Long Range	`EXP-20261006-001`	**63.55%**	10,000
Carry	`EXP-20261006-003`	62.77%	5,000
Halt	`EXP-20261006-005`	62.67%	10,000
Baseline (Local Only)	`EXP-20261006-001`	62.60%	10,000
Flat Baseline	`EXP-20261006-001`	62.49%	15,000

Broadcasting the goal cell into the spatial perception field (`long_range`) yields the highest accuracy among ablations (63.55%), confirming that conditioning local trace steps on global goal coordinates prevents underreaching in long agent trajectories.

5 Discussion and Limitations

While 1D NCAs demonstrate clear advantages in parameter efficiency and inference throughput, several limitations remain. First, a closed vocabulary of 13 tool names does not capture arbitrary nested arguments or dynamic APIs. Second, on local private developer traces, the highly skewed terminal prior presents an obstacle for non-carry architectures. Extending 1D NCAs to multi-head argument generation represents an important avenue for future research.

6 Conclusion

We have presented TextNCA, a one-dimensional neural cellular automaton for fast, parameter-efficient next-tool routing in agentic systems. By enforcing a 150k training floor, we demonstrated that spatial cellular update rules surpass flat linear layers and small LLMs on structured execution traces. This establishes cellular automata as a viable, lightweight routing substrate for autonomous multi-agent environments.

Code and Model Availability

Four public artifacts are made available:

1. **TextNCA Model:** `cap-jmk-real/textnca-next-tool`
2. **Baseline Model:** `cap-jmk-real/baseline-next-tool`
3. **Specialist Student:** `cap-jmk-real/specialist-python`
4. **Dataset:** `cap-jmk-real/hermes-next-tool-mix`

All training recipes and experiment cards (`EXP-*`) are maintained in the accompanying repository.

References

[1] A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin. Growing neural cellular automata. *Distill*, 5(2):e23, 2020.

[2] A. Mittal et al. TextNCA: Exploring Cellular Automata for Language Representation. *arXiv preprint arXiv:2603.01124*, 2026.